# 神经网络学习笔记

嵌入、MLP、训练与评估

第 3.1–3.7 节（P14–19） 讨论整理初稿 v1 2026-11-04

**范围与约定**。依据 Aaron Mazel-Gee 讲义 P14–19，并补入讨论中的推导。重要术语附英文；全文采用行向量。详细来源见末页。

## 1 数据处理：从名字到预测样本

对应原文：第 3.1–3.2、3.6 节。

### 1.1 字符集合与 n-gram

字符集合，也称本任务的**词表**（vocabulary），记为

$$\text{Chars} = \{*, a, b, \ldots, z\}, \qquad v := |\text{Chars}| = 27.$$

`*` 是**边界符号**（boundary token），用于填充名字开头并标记名字结束。

一个 $n$-gram 是连续 $n$ 个字符组成的片段。将它分成**上下文**（context）与紧随其后的**目标字符**（target）：

$$(s, x), \qquad s = (s_1, \ldots, s_k) \in \text{Chars}^k, \quad x \in \text{Chars}, \qquad \boxed{k = n - 1}.$$

上下文长度 $k$ 也叫 *block size*。**四元模型**（quadgram model）中 $n = 4$、$k = 3$。

### 1.2 按名字划分，再用滑动窗口拆分

先把完整名字划分到训练集、验证集和测试集，再分别拆成样本。开头补 $k$ 个 `*`，末尾加一个 `*`。例如，`emma` 在四元模型中产生：

| 上下文 $s$ | 目标 $x$ |
|---|---|
| `***` | `e` |
| `**e` | `m` |
| `*em` | `m` |
| `emm` | `a` |
| `mma` | `*` |

长度为 $q$ 的名字贡献 $q + 1$ 条样本，包括最后一次结束符号的预测。

### 1.3 允许重数

**数据集**（dataset）应理解为允许重复的列表或**多重集**（multiset）。`lemma` 和 `emma` 都会贡献 `emm`→`a`；若都属于训练集，这条样本就在训练损失中计两次。

记 $n(s, x)$ 为样本 $(s, x)$ 在某份数据中的出现次数，$T$ 为其样本总数，则

$$T = \sum_{s \in \text{Chars}^k} \sum_{x \in \text{Chars}} n(s, x).$$

**易错点**

1. 同一个名字贡献的样本留在其所属划分中；不同划分仍可出现相同字符组合。
2. 重复样本计重数。去重会改变经验频率及训练目标。

# 2 嵌入向量：可学习的字符表示

对应原文：第 3.1 节。

## 2.1 行向量与独热编码

全文采用**行向量约定**（row-vector convention）：

$$\mathbf{u} = (u_1, \dots, u_d) \in \mathbb{R}^{1\times d}, \qquad (1\times d)(d\times q) = (1\times q).$$

粗体小写字母表示向量，大写字母表示矩阵；上标 $\mathsf{T}$ 表示**转置**（transpose）。

固定一个一一对应的字符编号函数

$$\mathrm{ind} : \mathrm{Chars} \longrightarrow \{1, \dots, v\}.$$

字符 $t$ 的**独热编码**（one-hot encoding）是行向量

$$\mathbf{e}_{\mathrm{ind}(t)} \in \mathbb{R}^{1\times v},$$

仅对应位置为 1，其余位置为 0。它能区分字符，但没有预设不同字符之间的相似性。

## 2.2 嵌入矩阵与查表

选择**嵌入维数**（embedding dimension）$m$，为每个字符分配一个 $m$ 维行向量。把 $v$ 行排成**嵌入矩阵**（embedding matrix）：

$$\boxed{C \in \mathbb{R}^{v\times m}}, \qquad \boxed{\mathbf{c}_t = \mathbf{e}_{\mathrm{ind}(t)} C \in \mathbb{R}^{1\times m}}.$$

上述乘法取出 $C$ 对应字符的那一行，也可直接用**嵌入查表**（embedding lookup）实现。

- $m$ 是人为选择的**超参数**（hyperparameter）。
- $C$ 的 $vm$ 个元素是通过训练学习的**参数**（parameters）。

坐标通常从随机值开始调整，不必预先指定每个坐标的含义。

## 2.3 上下文向量：按顺序拼接

对上下文 $s = (s_1, \dots, s_k)$，进行**拼接**（concatenation）：

$$\boxed{\mathbf{u}(s) = (\mathbf{c}_{s_1}, \dots, \mathbf{c}_{s_k}) \in \mathbb{R}^{1\times km}}.$$

例如，$m = 5$、$k = 3$ 时，三个五维向量拼成一个十五维行向量。记号

$$(\mathbb{R}^5)^3 \cong \mathbb{R}^{15}$$

表示按固定顺序展开与拆分坐标得到的线性同构。

同一字符在不同上下文中使用同一行向量。这种**参数共享**（parameter sharing）使不同样本共同学习字符表示，并为泛化提供可能。

**易错点**

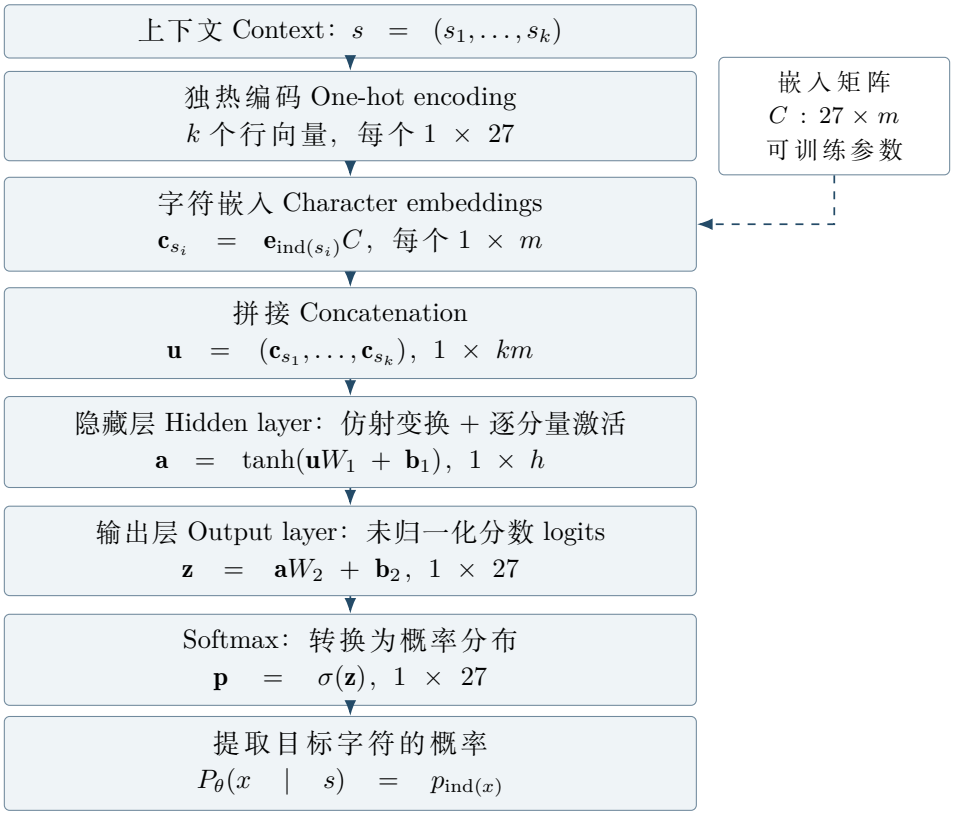
1. $C$ 有 $v$ 行、$m$ 列。矩阵元素是参数，维数 $m$ 是超参数。
2. 拼接不是相加；它保留位置和顺序，输出维数为 $km$。
3. 增大 $k$ 不扩大嵌入表，但扩大第一隐藏层的输入维数。
4. 若给 $v^k$ 个完整上下文各分配一个独立的 $m$ 维向量，仍需 $v^k m$ 个参数。原文脚注 47 的相关计数有误。

# 3 多层感知机与训练

对应原文：第 3.2–3.5 节。

## 3.1 模型架构与前向传播

**多层感知机**（multilayer perceptron, MLP）通过仿射变换和激活函数，把输入变成输出分数。以下是原文的单隐藏层结构，$v = 27$；沿箭头计算称为**前向传播**（forward propagation）。




**嵌入矩阵 $C$ 是参与计算的参数，不是输入数据转换后生成的矩阵。**

| 参数 | 形状 | 元素数量 |
|---|---|---|
| 嵌入矩阵 $C$ | $v \times m$ | $vm$ |
| 隐藏层权重 $W_1$ | $km \times h$ | $kmh$ |
| 隐藏层偏置 $\mathbf{b}_1$ | $1 \times h$ | $h$ |
| 输出层权重 $W_2$ | $h \times v$ | $hv$ |
| 输出层偏置 $\mathbf{b}_2$ | $1 \times v$ | $v$ |

总参数量为 $vm + kmh + h + hv + v$。原文取 $v = 27, k = 3, m = 2, h = 100$，共 3481 个参数。

## 3.2 隐藏层与输出概率的公式

### 一般的隐藏层

对含 $H$ 个**隐藏层**（hidden layers）的模型，令

$$\mathbf{a}^{(0)} = \mathbf{u}(s), \qquad d_0 = km.$$

第 $\ell$ 层计算

$$\boxed{\mathbf{a}^{(\ell)} = f_\ell\Big(\mathbf{a}^{(\ell-1)} W_\ell + \mathbf{b}_\ell\Big)}, \qquad \ell = 1, \dots, H.$$

$W_\ell$ 是**权重矩阵**（weight matrix），$\mathbf{b}_\ell$ 是**偏置向量**（bias vector），$f_\ell$ 是逐分量作用的**激活函数**（activation function）。形状为

$$\mathbf{a}^{(\ell-1)} \in \mathbb{R}^{1\times d_{\ell-1}}, \quad W_\ell \in \mathbb{R}^{d_{\ell-1}\times d_\ell},$$
$$\mathbf{b}_\ell \in \mathbb{R}^{1\times d_\ell}, \qquad \mathbf{a}^{(\ell)} \in \mathbb{R}^{1\times d_\ell}.$$

隐藏层数量、各层宽度（layer widths）和激活函数可以作为超参数。第一层输入维数由 $km$ 决定。

原文使用**双曲正切函数**（hyperbolic tangent）：

$$\tanh(t) = \frac{e^t - e^{-t}}{e^t + e^{-t}} \in (-1, 1).$$

若没有非线性激活，多次**仿射变换**（affine transformations）的复合仍是仿射变换。

### 输出层与 softmax

**输出层**（output layer）先给出 $v$ 个未归一化分数，称为 logits：

$$\mathbf{z} = \mathbf{a}^{(H)} W_{\text{out}} + \mathbf{b}_{\text{out}} \in \mathbb{R}^{1\times v}, \quad W_{\text{out}} \in \mathbb{R}^{d_H\times v}, \quad \mathbf{b}_{\text{out}} \in \mathbb{R}^{1\times v}.$$

Softmax 把它们变为**概率分布**（probability distribution）：

$$\boxed{p_j = \frac{e^{z_j}}{\sum_{q=1}^{v} e^{z_q}}}, \qquad p_j > 0, \qquad \sum_{j=1}^{v} p_j = 1.$$

目标字符 $x$ 的概率为 $P_\theta(x \mid s) = p_{\text{ind}(x)}$；$\theta$ 统称全部嵌入、权重和偏置参数。

Softmax 具有**平移不变性**（translation invariance）：

$$\sigma(\mathbf{z} + c\mathbf{1}) = \sigma(\mathbf{z}).$$

所有分数相同时得到均匀分布；所有分数同时加上常数不改变概率。

### 易错点

1. 偏置维数与输出维数相同。例如 $15 \times 80$ 的权重矩阵配 $1 \times 80$ 的偏置。
2. Softmax 输出概率；$\log p_j$ 才是**对数概率**（log-probability）。
3. 最终候选字符数固定为 $v = 27$；隐藏层宽度可以选择，输出维数由词表决定。

## 3.3 似然与平均负对数似然

### 概率与似然的视角

设数据为 $D = ((s_i, x_i))_{i=1}^{T}$。固定数据、改变参数 $\theta$，得到**似然函数**（likelihood function）：

$$\mathcal{L}(\theta; D) = \prod_{i=1}^{T} P_\theta(x_i \mid s_i).$$

对名字生成模型，名字内部的乘积来自**概率链式法则**（chain rule of probability），其中模型仅保留最近 $k$ 个字符作为条件；不要求相邻字符独立。将不同名字视为独立样本后，再相乘得到数据集似然。

| 上下文 | 目标 | 模型给正确字符的概率（演示值） |
|---|---|---|
| `***` | `e` | 0.1 |
| `**e` | `m` | 0.5 |
| `*em` | `m` | 0.5 |
| `emm` | `a` | 0.8 |
| `mma` | `*` | 0.5 |

模型生成完整名字 `emma` 并在此结束的概率为

$$0.1 \times 0.5 \times 0.5 \times 0.8 \times 0.5 = 0.01.$$

计算时，输入已知上下文，读取正确字符的概率即可，不需要真的抽样生成名字。

### 把最大化乘积改成最小化损失

对数严格递增，且把乘积变成求和。因此最大化似然，等价于最小化**平均负对数似然**（average negative log-likelihood, NLL）：

$$\boxed{L_D(\theta) = -\frac{1}{T} \sum_{i=1}^{T} \log P_\theta(x_i \mid s_i)}.$$

这里 log 是自然对数，$L_D$ 是**损失函数**（loss function）。除以 $T$ 后，每条样本等权计入平均值。上例平均损失为

$$-\frac{1}{5} \log(0.01) \approx 0.9210.$$

按照重复次数合并，同一个公式变为

$$L_D(\theta) = -\frac{1}{T} \sum_{s} \sum_{x} n(s, x) \log P_\theta(x \mid s).$$

若只有 `lemma`、`emma` 两个名字，共 11 条样本；两次 `emm`→`a` 的损失贡献为

$$-\frac{2}{11} \log P_\theta(a \mid \texttt{emm}).$$

### 易错点

1. 固定参数、让数据变化时讨论数据的概率；固定数据、让参数变化时讨论似然。
2. 似然不是“参数为真的概率”，也不需要对参数求和等于 1。
3. 每条样本取出的是目标字符对应的概率；重复样本在乘积中重复出现，在对数和中重复计数。

### 3.4 极大似然与经验频率

**伯努利例子**

设 $X_1, \dots, X_{M+N}$ 独立同分布，服从**伯努利分布**（Bernoulli distribution）：

$$P_p(X_i = 1) = p, \qquad P_p(X_i = 0) = 1 - p.$$

观察到的有序数据含 $M$ 个 1 和 $N$ 个 0，且 $M, N > 0$。似然与**对数似然**（log-likelihood）为

$$\mathcal{L}(p) = p^M (1-p)^N, \qquad \log \mathcal{L}(p) = M \log p + N \log(1-p).$$

对 $p$ 求导并令其为零：

$$\frac{d}{dp} \log \mathcal{L}(p) = \frac{M}{p} - \frac{N}{1-p} = 0 \quad \Longrightarrow \quad \boxed{\hat{p} = \frac{M}{M+N}}.$$

由于

$$\frac{d^2}{dp^2} \log \mathcal{L}(p) = -\frac{M}{p^2} - \frac{N}{(1-p)^2} < 0,$$

这是唯一最大值点。**极大似然估计**（maximum likelihood estimate, MLE）等于样本中 1 的**经验频率**（empirical frequency）。

若只观察计数而不观察顺序，似然多出组合系数 $\binom{M+N}{M}$；它与 $p$ 无关，不改变最优参数。

**为什么最小损失可能大于零**

令 $q = M/(M+N)$，最小平均负对数似然为

$$-q \log q - (1-q) \log(1-q) > 0 \qquad (0 < q < 1).$$

它是伯努利分布的**熵**（entropy）。同一输入对应不同结果时，无法把每个结果的概率同时设为 1。

对于固定上下文 $s$，若可自由选择输出分布，最优概率同样等于经验频率：

$$\hat{p}(x \mid s) = \frac{n(s,x)}{\sum_y n(s,y)}.$$

若有字符从未出现，最优分布会把它的概率设为零；有限 logits 的 softmax 只能逼近这种边界分布。

**易错点**

1. 相同上下文的不同目标不会获得独立的输出分布；模型只能给这个上下文一份分布。
2. 神经网络在不同上下文之间共享参数，未必能同时精确实现各自的经验频率。
3. 损失非零不一定意味着训练失败，也可能反映目标分布本身的非确定性。

### 3.5 随机梯度下降：用小批量估计梯度

将全部参数按固定顺序展开为行向量 $\boldsymbol{\theta} \in \mathbb{R}^{1\times P}$，其中 $P$ 是参数总数；**梯度**（gradient）也按相同顺序排成行向量。

第 $i$ 条训练样本的损失与完整训练损失为

$$\ell_i(\boldsymbol{\theta}) = -\log P_{\boldsymbol{\theta}}(x_i \mid s_i), \qquad L_{\text{train}}(\boldsymbol{\theta}) = \frac{1}{T}\sum_{i=1}^{T} \ell_i(\boldsymbol{\theta}).$$

求导的线性性给出

$$\nabla L_{\text{train}}(\boldsymbol{\theta}) = \frac{1}{T}\sum_{i=1}^{T} \nabla \ell_i(\boldsymbol{\theta}).$$

**反向传播**（backpropagation）利用链式法则计算导数，包括嵌入矩阵的导数。

**随机梯度下降**（stochastic gradient descent, SGD）每步均匀、有放回地抽取 $b$ 条样本，编号为 $I_1, \ldots, I_b$。这批样本称为**小批量**（minibatch），$b$ 为**批量大小**（batch size）。计算

$$\widehat{L}_B(\boldsymbol{\theta}) = \frac{1}{b}\sum_{j=1}^{b} \ell_{I_j}(\boldsymbol{\theta}), \qquad \mathbf{g}_B(\boldsymbol{\theta}) = \frac{1}{b}\sum_{j=1}^{b} \nabla \ell_{I_j}(\boldsymbol{\theta}),$$

再以**学习率**（learning rate）$r > 0$ 更新参数：

$$\boxed{\boldsymbol{\theta}_{\text{new}} = \boldsymbol{\theta} - r\mathbf{g}_B(\boldsymbol{\theta})}.$$

#### 固定参数时的无偏性

因为每个 $I_j$ 都在 $\{1, \ldots, T\}$ 中均匀抽取，

$$\mathbb{E}_B[\nabla \ell_{I_j}(\boldsymbol{\theta})] = \frac{1}{T}\sum_{i=1}^{T} \nabla \ell_i(\boldsymbol{\theta}).$$

因此批量梯度是**无偏估计**（unbiased estimator）：

$$\boxed{\mathbb{E}_B[\mathbf{g}_B(\boldsymbol{\theta})] = \nabla L_{\text{train}}(\boldsymbol{\theta})}.$$

单次估计仍有波动，不保证每步都降低完整训练损失。较大的批量通常给出更稳定的估计，但单次更新需要处理更多数据。

#### 易错点

1. 不同批次的损失混合了样本差异与参数变化，不能只凭一次上升判断模型变差。
2. 大数定律用于固定模型参数后的重复抽样；训练中历次损失的平均值并非当前模型损失的直接估计。
3. 一次更新叫 *step / iteration*；完整遍历训练集通常叫一个 *epoch*。原文在 SGD 中把 step 也称作 epoch。
4. 原文的 *true loss* 指完整训练集损失，不是测试损失或未来数据上的理论期望损失。

## 3.6 学习率调整与衰减

### 方向正确，步幅也要合适

完整梯度下降满足

$$\boldsymbol{\theta}_{\text{new}} = \boldsymbol{\theta} - r\nabla L(\boldsymbol{\theta}), \qquad \|\boldsymbol{\theta}_{\text{new}} - \boldsymbol{\theta}\| = r\|\nabla L(\boldsymbol{\theta})\|.$$

实际移动距离同时取决于学习率与梯度大小。一阶近似为

$$L(\boldsymbol{\theta} + \Delta\boldsymbol{\theta}) \approx L(\boldsymbol{\theta}) + \nabla L(\boldsymbol{\theta})\,\Delta\boldsymbol{\theta}^{\mathsf{T}}.$$

代入负梯度更新，近似下降量为 $r\|\nabla L\|^2$。但这只是局部近似，步幅过大时高阶项不可忽略。

### 二次函数例子

对 $f(t) = at^2/2$、$a > 0$，梯度下降为

$$t_{j+1} = (1 - ra)t_j.$$

对于一般非零初值，收敛需要

$$|1 - ra| < 1 \quad \Longleftrightarrow \quad \boxed{0 < r < 2/a}.$$

当 $a = 1$ 时：

| 学习率 $r$ | 每步乘数 $1 - r$ | 行为 |
|---|---|---|
| 0.1 | 0.9 | 从同一侧逐渐接近零 |
| 1 | 0 | 一步到零 |
| 1.5 | $-0.5$ | 交替换侧，幅度缩小 |
| 2 | $-1$ | 交替换侧，幅度不变 |
| 2.5 | $-1.5$ | 交替换侧，幅度增大 |

### 搜索范围与学习率衰减

原文先测试学习率量级，再以**对数尺度**（logarithmic scale）从 $10^{-3}$ 逐渐增大到 1，观察损失趋势，最终选择 $r = 0.1$。这是一种经验搜索方法，不证明找到了全局最优学习率。

**学习率衰减**（learning rate decay）在训练后期降低 $r$。原文从 0.1 降到 0.01 后损失继续改善。较小步幅便于细调，也减轻随机梯度造成的来回移动，类似磨光材料时逐渐换用更细的砂纸。

### 易错点

1. 学习率不是实际移动距离；梯度大小同样影响步幅。
2. 跨过最小值、出现左右摆动并不必然失败，关键在于幅度是否缩小。
3. SGD 自带抽样波动，单次损失上升不足以证明学习率过大。
4. 合理学习率依赖损失曲率、批量大小及训练阶段，不存在这里推导出的通用数值阈值。

# 4 过拟合、数据划分与模型选择

对应原文：第 3.3、3.6–3.7 节。

## 4.1 拟合与泛化

**拟合**（fitting）训练数据，是提高已用于训练的数据的预测概率。**泛化**（generalization），是对未参与训练、来自同类来源的数据也能做出良好预测。

**过拟合**（overfitting）表现为模型过度适应训练数据的具体细节，训练表现很好，新数据表现却较差。参数多、数据少会增加这种风险，但仅凭参数数量不能证明过拟合。

直觉上，高次多项式可以穿过所有观测点，但也可能把样本中的偶然波动一并拟合，导致新位置上的预测较差。

### 原文的小数据集实验

原文先只用 `emma`、`olivia`、`ava`、`isabella`、`sophia` 五个名字产生的 32 条样本训练，损失降至约 0.2552。

五条起始样本的上下文同为 `***`，目标却是五个不同字符。它们的负对数似然总和至少为 $5\log 5$；其余项非负，因此平均损失满足

$$\boxed{L \geq \frac{5\log 5}{32} \approx 0.2515}.$$

结果已接近这个下界，说明对小训练集拟合充分，但没有直接告诉我们新名字上的表现。

## 4.2 训练集、验证集与测试集

| **数据划分**（data split） | 用途 |
|---|---|
| 训练集（training set） | 计算梯度，更新模型参数 |
| 验证集（validation / development set） | 比较超参数、训练时长等方案 |
| 测试集（test set） | 方案确定后的最终评估 |

原文比例为 80%、10%、10%；这不是固定要求。

**模型选择**（model selection）的流程为：

1. 固定数据划分，为一组候选超参数初始化模型。
2. 仅用训练集，通过 SGD 更新参数。
3. 固定模型参数，计算验证集的平均损失。
4. 比较候选方案，根据验证表现选择。
5. 方案确定后，使用测试集做最终评估。

验证可在训练中定期进行，但验证过程不更新参数。验证集虽然不用于求梯度，仍通过方案选择影响最终模型。

### 4.3 泛化差距与调参判断

对同一个已训练模型，定义验证与训练损失之差：

$$\boxed{\Delta_{\text{dev}}(\theta) = L_{\text{dev}}(\theta) - L_{\text{train}}(\theta)}.$$

它是**泛化差距**（generalization gap）的一种经验估计，针对具体参数 $\theta$，而非仅针对一组超参数。同一组超参数经过不同初始化与 SGD 抽样，也可能得到不同模型。

评估时须同时看两件事：

- **损失的绝对大小：**预测本身做得多好。
- **训练与验证的差距：**训练表现能否延续到新数据。

差距小可能意味着两者都好，也可能意味着两者都差。**欠拟合**（underfitting）不能仅由“小差距”断言，还要结合损失水平、训练情况，以及扩大模型后的验证表现。

| 模型 | 训练损失 | 验证损失 | 差距 |
|---|---|---|---|
| A | 0.30 | 2.80 | 2.50 |
| B | 1.00 | 2.10 | 1.10 |

若目标是预测新名字，选择验证损失较低的 B。A 显示更明显的过拟合迹象；不能由此断言 B 完全没有过拟合。

#### 第 3.7 节中的模型容量调整

原文固定 $v = 27$、$k = 3$，逐步调整隐藏层宽度 $h$ 和嵌入维数 $m$：

| $m$ | $h$ | 参数数量 | 训练损失 | 验证损失 |
|---|---|---|---|---|
| 2 | 100 | 3481 | 2.3313 | 2.3318 |
| 2 | 300 | 10281 | 2.2382 | 2.2464 |
| 10 | 200 | 11897 | 2.1674 | 2.1939 |

这些实验的训练安排并不完全相同，因此不是严格控制变量的比较。它们表明相应训练方案下验证损失有所改善，而不是说明“泛化差距越大越好”。

#### 易错点

1. 比较模型应使用同一评估集和同一损失定义；不能把小训练集损失直接与另一数据集损失比较。
2. 验证集通过超参数选择影响模型；反复根据测试结果调参，会使测试集承担验证集的角色。
3. 泛化差距比较同一个模型在不同数据划分上的表现，不是两个超参数方案之间的差。
4. 训练与验证损失接近，不足以单独证明欠拟合；还需观察损失水平与进一步实验。
5. 调参目标是降低验证损失。训练损失下降而验证损失上升，不能仅凭训练改善就保留新模型。

**来源说明**。本笔记为学习讨论整理。主要依据 Aaron Mazel-Gee，*“Neural Networks: Zero to Hero”: A Mathematician's Companion*，2024-04-09 草稿，第 3.1–3.7 节，PDF 第 14–19 页。伯努利估计推导、损失下界、二次函数例子及记号澄清包含讨论中的补充。